

PRODUCT REQUIREMENTS · DRAFT V1.1 · ZERO-COST BUILD

CompareFurniture

A price-comparison layer for furniture and appliance rental in Bengaluru. One search, every provider, honest totals — so nobody has to open six tabs to find out what a bed actually costs for a year. Built as a personal project, on infrastructure that costs nothing to run.

STATUS

Draft for review

RUNNING COST

₹0 / month

DATE

25 Aug 2026

MARKET

Bengaluru only

PHASE 1 TARGET

4 categories, 3 providers

STACK

Java 21 · Actions cron · static site

► CONTENTS

01 — SUMMARY

What we are building

A search box, a comparison table, and a number the user can trust. Everything else in this document exists to make those three things correct.

A user tells us their city, how long they are renting for, and what they need. We return every provider's price for each item, the cheapest way to buy the whole basket, and the true cost over the rental period — rent plus deposit plus delivery plus installation, with refundable money separated from money that is gone.

The product is thin on features and heavy on data integrity. The hard parts are not the UI; they are keeping eleven independent providers' prices current without hammering their sites, normalizing "Single Door Fridge" and "Samsung 192L" into the same comparable row, and never showing a number

we cannot stand behind.

SCOPE CHANGE RECOMMENDED BEFORE BUILD STARTS

The proposed Phase 1 provider set includes two providers whose `robots.txt` currently disallows exactly the listing paths we would need to read. Section 14 has the evidence per provider. The recommendation is to ship Phase 1 on **RentoMojo**, **Rentickle** and **Guarented**, and route **Cityfurnish** and **Furlenco** through partnership or a manual-entry adapter until a feed exists.

The promise

Compare furniture and appliance rental prices across Bengaluru in one place.

The question we answer

"What is the cheapest way to rent the things I need, for the months I need them?"

The line we don't cross

No fabricated prices, no stale data presented as current, no paid placement inside a "cheapest" ranking.

The budget

₹0 per month, permanently. Not "cheap" — zero. That constraint does more design work than any feature in this document.

WHAT THE BUDGET REMOVES

An always-on server, a managed database and a Redis instance were the only line items with a price. All three existed to serve concurrency this project will never have. Removing them takes the distributed lock, the TTL orchestrator, the admin authentication layer and the API rate limiter with them — roughly a third of the original build, deleted rather than descope.

What survives is everything that made the product correct: adapter isolation, normalization, the cost model, and the freshness rules. Those are engineering problems, not scaling problems, and they are free.

Every line item, and what it costs

The target is not a small bill. It is no bill, no card on file, and no service that can start charging because a threshold moved.

NEED	FREE SERVICE USED	INCLUDED	OUR ESTIMATED USE
Scheduled compute for scraping	GitHub Actions, standard runner	2,000 min/mo private unlimited on public repos	~480 min/mo 24% OF BUDGET
Headless browser	Playwright on the Actions Ubuntu runner	included	₹0
Data storage	The Git repository itself	soft limit ~1 GB	a few MB/yr of JSON
Price history	Git commit history	included	₹0
Job mutex	Actions concurrency group	included	₹0
Site hosting	Cloudflare Pages or Vercel Hobby	500 builds/mo · generous transfer	~60 builds/mo
Failure alerting	GitHub Actions failure email	included	₹0
Domain	*.pages.dev / *.vercel.app subdomain	free	₹0
Custom domain	the only thing on this page that costs money	—	~₹1,000/yr — DECLINED

Two conditions on the free tiers

C-1	Vercel Hobby is non-commercial only. Their docs state it plainly: the Hobby plan “restricts users to non-commercial, personal use only”. The moment an affiliate link appears on the site, the project needs Vercel Pro at \$20/mo or a move to Cloudflare Pages, which carries no such clause. Building on Cloudflare Pages from the start avoids the choice entirely — see §22.
C-2	GitHub disables scheduled workflows after 60 days of repository inactivity.

Commits made by the workflow’s own token may not reset that clock. The cron therefore needs a keepalive — a monthly manual commit, or a second workflow that touches the repo — or the site silently stops updating while still looking healthy.

THE HONEST VERSION OF “FREE”

Free tiers are a business decision by someone else, and they change. The mitigation is not to find better free tiers — it is that nothing here is hard to move. The pipeline is a Java CLI and the data is JSON files in a repo; if Actions changes terms, the same CLI runs on any machine with a cron entry, including a laptop. Portability is the actual insurance policy, not the current price list.

03 – PROBLEM

Six tabs and no answer

Furniture rental in Bengaluru is a mature, fragmented market. A person setting up a flat has to visit RentoMojo, Cityfurnish, Furlenco, Rentickle and Guarented separately, re-enter the same tenure on each, and then do arithmetic that none of the sites do for them.

Why the arithmetic is the actual problem

Headline monthly rent is not comparable across providers, because the rest of the cost structure differs and is disclosed late — often only at checkout.

WHAT VARIES	WHY IT BREAKS NAIVE COMPARISON
Security deposit	Refundable, but locks up cash. A ₹500/mo item with a ₹10,000 deposit beats a ₹550/mo item with ₹2,000 only at long tenures.
Tenure pricing	Monthly rate usually falls with a longer commitment. A 3-month quote and a 24-month quote are different products.
Delivery & installation	Sometimes free, sometimes per-item, sometimes waived above a basket value.
Basket effects	Bundle discounts and free-delivery thresholds mean the cheapest

Serviceability	item-by-item pick is not always the cheapest basket.
	A price is meaningless if the provider does not deliver to Whitefield or Electronic City.

The result is that people either pick on brand recognition, or spend an evening in a spreadsheet. We are building the spreadsheet, once, correctly.

04 - GOALS

Goals and non-goals

Goals

- **Comparable prices.** Normalize heterogeneous catalogues into canonical products so a row genuinely compares like with like.
- **Honest totals.** Show estimated total cost for the chosen tenure, with refundable deposit stated separately from sunk cost.
- **Two answers, not one.** Cheapest cross-provider basket *and* cheapest single-provider basket, with the delta between them.
- **Fresh, attributable data.** Every price carries a provider, a source URL and a checked-at timestamp, surfaced in the UI.
- **Fail partially, never wholly.** One broken provider degrades one column, not the site.
- **Replaceable providers.** Adding or swapping a provider must not touch the comparison engine.

Non-goals

- **We are not a rental marketplace.** No checkout, no inventory, no delivery, no KYC. We hand the user off to the provider.
- **No cities beyond Bengaluru in v1.** The schema is multi-city; the product is not.
- **No accounts in Phase 1.** Comparison works anonymously.
- **No used/resale, no rent-to-own modelling, no B2B/office bulk quoting.**
- **No real-time price fetch on user request.** Prices are served from our store;

refresh is asynchronous.

- **No circumvention.** We do not defeat CAPTCHAs, anti-bot systems, logins or paywalls to obtain data (§14).

05 — USERS

Who this is for

All primary users share one trait: a **move-in date**. They are deciding under time pressure, usually within a week of taking possession of a flat.

The relocater

Moving to Bengaluru for a job, 12–24 month horizon, furnishing a 1BHK or 2BHK from zero. Highest basket value, most price-sensitive to deposit.

The bachelor flat-share

2–3 people splitting a flat. Needs multiples of the same item. Optimizes hard on monthly rent; tenure often uncertain.

The short-stay professional

3–6 months on a project. Deposit dominates their total cost; a low headline rent can be the worse deal.

The gap-filler

Owns most things, needs one appliance — a washing machine or a fridge. Single-item comparison, arrives from search.

Jobs to be done

- When I am furnishing a flat, I want one view of every provider's price, so I don't have to trust that the first site I opened was fair.
- When I compare two quotes, I want the deposit and fees folded into a single number for my actual tenure, so I can tell which is genuinely cheaper.
- When I find a good price, I want to know it is current, so I don't arrive at the provider's site and find a different number.
- When mixing providers saves little, I want to know that, so I can choose one company and one delivery date instead.

06 – METRICS

How I know it works

This is a personal project, so click-through and conversion targets would be theatre. The metrics that matter are the ones that tell me whether the data is true and the pipeline is alive — and every one of them is computable inside the build, with no analytics service.

LAYER	METRIC	TARGET	WHERE IT COMES FROM
Truth	Sampled prices matching the provider's own page	$\geq 97\%$	Manual spot-check, 20 items, monthly
Truth	Prices displayed with age under 12 h	$\geq 95\%$	Computed at build from <code>scrapedAt</code>
Liveness	Consecutive successful scheduled runs	$\geq 95\%$ of runs	<code>runs.json</code> , written every run
Liveness	Days since last successful run	≤ 1	Status page — the single number that says the project is still alive
Coverage	Canonical products priced by ≥ 2 providers	$\geq 70\%$	Computed at build
Coverage	Provider products auto-matched without review	$\geq 85\%$	Normalizer output
Cost	Actions minutes consumed per month	$< 50\%$ of quota	GitHub billing page

Coverage is still the metric that decides whether this is a comparison site or a list of links. With three providers the bar drops from three prices per row to two — below that, a row has nothing to compare.

07 – SEARCH

The search

The homepage does one thing: collect city, tenure and a basket. The hero states the promise and the form sits directly beneath it — no scroll required on a laptop.

FR-1.1	City. Defaults to Bengaluru and is the only selectable value in v1. The control is present and disabled-with-reason, not hidden, so the multi-city intent is legible.
FR-1.2	Tenure. 3 / 6 / 9 / 12 / 18 / 24 months plus Custom. Default 12. Tenure is a first-class query parameter, not a display filter — it changes which price rows are selected.
FR-1.3	Basket. Multi-select over 15 categories: bed, mattress, refrigerator, washing machine, sofa, wardrobe, study table, office chair, dining table, TV, air conditioner, microwave, air cooler, water purifier, other.
FR-1.4	Quantity. Each selected category carries a quantity (default 1). Two beds is the common case, not an edge case.
FR-1.5	Variant is optional at search time. A user picks "Bed"; they refine to "Queen, with storage" on the results page. Forcing variant choice up front costs completions.
FR-1.6	Shareable state. The full query serializes into the URL so a comparison can be sent to a flatmate.
FR-1.7	Popular comparisons. Below the form: bed, fridge, washing machine, mattress, 2BHK setup, bachelor setup — each a pre-filled query, and each an indexable landing page (\$22).

08 — RESULTS

Comparison results

Desktop gets a matrix: canonical products down the left, providers across the top, monthly price in the cells, a total row at the bottom. Mobile gets one card per product with providers ranked inside it. Same data, two shapes; the matrix does not survive a 390px viewport and should not be forced to.

FR-2.1	Cells show monthly price for the selected tenure, with a secondary line for the estimated total when the total-cost view is toggled on.
FR-2.2	The lowest price in each row carries a BEST PRICE badge. Ties are shown as ties, not arbitrarily ordered.
FR-2.3	A provider with no matching product shows an explicit NOT OFFERED , never a blank cell — blank reads as an error or a zero.
FR-2.4	A provider whose last successful refresh is older than 24h shows its column dimmed with CHECKED 28H AGO . A provider in hard failure shows the last known price with an explicit staleness label, or is dropped from the total with a footnote.
FR-2.5	The per-provider total row includes only items that provider actually stocks, and is annotated with how many of the requested items it covers (e.g. <i>3 of 4 items</i>). A 3-item total must never be visually comparable to a 4-item total without that label.
FR-2.6	Every price cell links to the provider's own listing for that product.
FR-2.7	A freshness line sits above the table: <i>Prices last checked 2 hours ago / Refreshing — showing latest available prices / Updated just now.</i>

DESIGN CONSTRAINT

Coverage gaps are the honest failure mode of a comparison site and must be designed for, not hidden. A cell is one of four states — priced, not offered, out of stock, or stale — and each has a distinct visual treatment.

09 — ENGINE

The cheapest-basket engine

This is the product's core computation and it must be deterministic, explainable, and independent of any provider code. It produces two answers side by side.

A. Cheapest cross-provider basket

For each requested item, pick the provider offering the lowest true cost for that item at the chosen tenure; sum. Because provider-level effects exist (free delivery above a basket threshold, bundle discounts), the greedy per-item pick is only correct once those effects are applied at basket level. The engine therefore evaluates candidate assignments and applies provider-level adjustments before comparing totals.

B. Cheapest single-provider basket

For each provider that stocks *all* requested items, compute the basket total with that provider's bundle rules; take the minimum. Providers with partial coverage are reported separately, never silently mixed into this figure.

C. The delta

Present the saving from splitting across providers explicitly. If it is small, say so — a ₹120/month saving spread over four deliveries, four delivery windows and four support relationships is a real trade-off the user should make consciously.

CHEAPEST COMBINATION · 12 MONTHS	
Queen bed — Provider A	₹550
Mattress — Provider B	₹349
Refrigerator — Provider A	₹499
Washing machine — Provider B	₹659
Total per month	₹2,057
vs. cheapest single provider	₹2,180
You save by splitting	₹123 / mo

Illustrative layout only. Every figure in production comes from the price store; no example price in this document may appear in code.

Requirements

FR-3.1	The engine reads only from the normalized price store. It has no knowledge of providers beyond their IDs and fee rules, and no network calls.
--------	---

FR-3.2	Ranking is by <i>estimated total cost over the selected tenure</i> , never by monthly rent alone (\$10).
FR-3.3	Every result is explainable: the response includes the per-item chosen provider, unit price, quantity and the adjustments applied.
FR-3.4	Items with no price from any provider are excluded from totals and reported as uncovered.
FR-3.5	Sponsored or featured status has zero weight in this computation. Ever. (\$22)

10 – COST MODEL

What a rental actually costs

One formula, applied identically to every provider, is what makes the comparison fair.

```
estimatedTotal = (monthlyPrice × tenureMonths)
                + deliveryFee
                + installationFee
                + otherMandatoryFees
                - discounts

refundableHeld  = securityDeposit
cashRequiredUpfront = securityDeposit + deliveryFee + installationFee + firstMonth
```

Deposit is deliberately outside `estimatedTotal` — it comes back. But it is cash the user does not have for the duration, so it gets equal billing in the UI as a separate, clearly labelled figure. Three numbers are always shown together: **monthly**, **estimated total (non-refundable)**, and **refundable deposit**.

FIELD	TREATMENT	SHOWN AS
Monthly rent	× tenure	Sunk
Security deposit	Held, returned at end	Refundable — separate line

Delivery fee	One-off, may be threshold-waived	Sunk
Installation fee	One-off, per item	Sunk
Other mandatory charges	One-off or recurring; must be labelled	Sunk
Discounts	Subtracted; source recorded	Reduces sunk
Relocation / early-closure charges	PHASE 2 Not modelled in v1; noted in provider detail if published	—

WHY THIS IS THE RANKING KEY

₹500/mo with a ₹10,000 deposit and ₹550/mo with a ₹2,000 deposit are the same headline story and different decisions. Over 3 months the deposit dominates; over 24 months the rent does. Ranking on monthly price alone would give the wrong answer to the short-stay user — one of our four core personas.

11 — DETAIL & CONTROLS

Product detail, filters, sorting

Product detail

Opening a cell shows the provider's actual listing as we hold it: name, image, brand, model, size, features, monthly price, deposit, delivery, installation, minimum rental period, availability, provider, last-checked timestamp, and a link out. The last-checked timestamp is not fine print — it is a primary trust signal and sits next to the price.

Filters

FILTER	VALUES
Category	The 15 categories in FR-1.3
Provider	Multi-select

Monthly price	Min / max
Tenure	3 / 6 / 9 / 12 / 18 / 24 / custom
Deposit	Max deposit — the filter short-stay users need most
Refrigerator attrs	Single / double door; capacity band
Washing machine attrs	Top / front load; capacity band
Bed attrs	Single / double / queen / king; storage yes-no
Freshness	Hide prices older than 24h

Sorting

- Cheapest monthly · Cheapest total for tenure · Lowest deposit · Lowest upfront cash · Recently updated
- **Best value** — the composite score below. It is never the default sort in v1; *cheapest total* is, because it is the one a user can verify by hand.

Best-value score

A single ordering over estimated total cost, adjusted for confidence.
Deterministic, documented, and shown with a "why this ranked here" breakdown on hover.

```
score = normalized(estimatedTotalForTenure)           // dominant term
      + w1 × normalized(cashRequiredUpfront)
      - w2 × freshnessConfidence(scrapedAt)           // stale data ranks lower
      - w3 × providerReliability(30d success rate)
```

Freshness enters the ranking rather than only the UI: a price we are less sure of should not win a comparison on a technicality.

Build my rental setup

Most users do not think in SKUs: they think "2BHK. two people. one year".

This feature converts a flat description into a basket and runs the standard engine on it. It is the highest-value acquisition surface and the natural landing page for relocation search traffic.

FR-4.1	Inputs: apartment type (1BHK / 2BHK / 3BHK), occupants (1 / 2 / 3+), tenure.
FR-4.2	Output: a suggested basket from a maintained template — e.g. 2BHK/2 people → 2 queen beds, 2 mattresses, 1 refrigerator, 1 washing machine, 1 sofa, 1 dining table, 2 wardrobes.
FR-4.3	The basket is fully editable before comparison. The template is a starting point, not an answer.
FR-4.4	Results show cheapest mixed-provider total, cheapest single-provider total, and the monthly saving between them.
FR-4.5	Templates live in <code>data/setups.yml</code> — editing one is a commit, not a code change, and the site rebuilds itself.

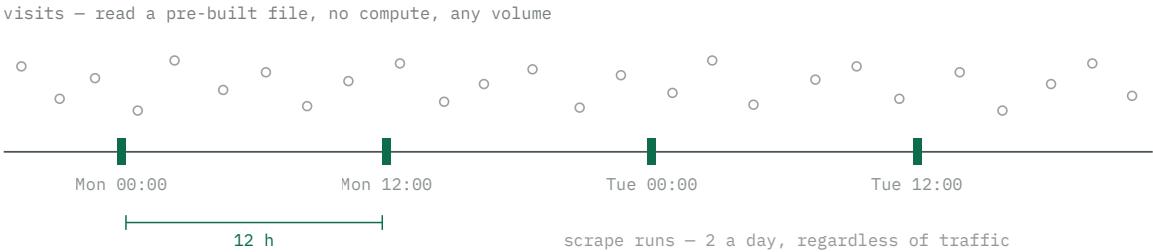
13 — FRESHNESS

The freshness contract

The original design had a 12-hour TTL, a request-triggered background refresh and a distributed lock to stop five hundred simultaneous visitors from starting five hundred scrapes. With no server, that entire mechanism collapses into one line of YAML.

```
on:
  schedule:
    - cron: "0 */12 * * *"      # the TTL, expressed as a schedule
concurrency:
  group: scrape                # the distributed lock, expressed as a policy
  cancel-in-progress: false
```

Refresh is no longer something a user can trigger, so there is nothing to stampede. Reads and writes were decoupled by construction the moment the site became static.



Traffic and refresh cadence are fully independent. The lock existed to protect providers from our own users; with a static site there is no path from a visit to a request against a provider at all.

Rules

FR-5.1	The pipeline runs on a 12-hour cron. Every price record carries <code>scrapedAt</code> , written by the run that produced it.
FR-5.2	Providers are refreshed independently within a run. One provider failing does not abort the others, and each writes its own <code>scrapedAt</code> .
FR-5.3	<code>concurrency.group</code> guarantees one run at a time. A manual <code>workflow_dispatch</code> and the cron share the group, so a hand-triggered run cannot collide with a scheduled one.
FR-5.4	A failed provider must not commit an empty result. The run writes the new file only for providers that succeeded; the previous committed values survive untouched and their age keeps climbing. This is what surfaces failure honestly, and it is the single most important rule in the pipeline.
FR-5.5	Scheduled Actions are best-effort and can be delayed under platform load. The UI therefore reports actual data age, never “last updated 12 hours ago” inferred from the schedule.
FR-5.6	Freshness is computed in the browser from <code>scrapedAt</code> against the current time — not baked in at build, which would let a cached page claim a price is fresh forever.

Trust rules for displayed data

AGE OF PRICE	UI TREATMENT
< 12 h	CHECKED 3H AGO — normal presentation
12–24 h	Normal presentation; a run is due or in flight

> 24 h	PRICE MAY BE OUTDATED — de-emphasized, excluded from “best price” badges
> 72 h	NOT VERIFIED — excluded from totals and rankings; product still listed with a link out

NON-NEGOTIABLE

Never fabricate a price. Never show an unavailable product as available. Never present stale data as current. Every price record must carry `provider` , `providerUrl` and `scrapedAt` — a record missing any of the three fails the build rather than reaching the site.

14 — PROVIDERS

Provider integration and its constraints

Each provider is an independent, replaceable adapter behind one interface. No provider may assume it has an API, and no provider's failure may be visible outside its own column.

Does an official API exist? No.

Checked on 25 Aug 2026: none of the six major providers publishes a developer portal, API documentation, or a partner data API, and no public affiliate product feed was found for any of them. Every integration in Phase 1 is therefore either a permitted crawl or a manual sheet. Partnership outreach is still worth doing — it is the only route that turns a fragile adapter into a stable one — but the plan must not depend on it.

NOT AN API

Two of the crawlable sites render prices client-side, which means their front ends call internal JSON endpoints that are visible in devtools. These are **not** APIs: undocumented, unversioned, unlicensed, and changeable without notice — and Rentickle’s `robots.txt` explicitly disallows `/api` . Reading them is scraping with less warning when it breaks, and it must not be recorded as `integration_type = API` . Render the permitted page in a

browser instead, or negotiate a real feed.

Extraction preference order

- 1. **Official API** — always preferred, always worth asking for. None exists today for any provider.
- 2. **Partner or affiliate feed** — comes with commercial alignment, and is the outcome partnership outreach should aim for. None found publicly today.
- 3. **Published structured data** — JSON-LD `Product / Offer` markup and sitemaps, where crawling that path is permitted.
- 4. **Server-rendered HTML** — permitted paths only.
- 5. **Headless browser (Playwright)** — only where prices render client-side and the path is permitted. Assume Jsoup alone is insufficient for several of these sites.
- 6. **Manual entry** — an admin-maintained price sheet with the same freshness metadata, used as an explicit, labelled fallback.

Observed crawl posture

`robots.txt` checked directly on 25 Aug 2026. This is a snapshot of a file that changes without notice — the adapter must re-fetch and honour it at runtime, and this table is documentation, not the enforcement mechanism.

PROVIDER	LISTING / PRODUCT PATHS UNDER USER-AGENT: *	VERDICT	ROUTE
RentoMojo	Allowed. Disallows only admin, user, checkout, order-summary and blog tags. Publishes per-city sitemaps including Bengaluru.	CRAWLABLE	Structured data / SSR HTML
Rentickle	Allowed. City directories including Bengaluru explicitly permitted. Disallows <code>/catalog</code> , <code>/catalogsearch</code> , <code>/api</code> , <code>/_next</code> and all query-string URLs (<code>/*?</code>).	CRAWLABLE	SSR HTML, clean paths only
Guarented	Allowed. Disallows only cart, checkout, dashboard, not-found and tracking-parameter URLs.	CRAWLABLE	SSR HTML, Playwright
Cityfurnish	Disallows <code>/shop</code> , <code>/things/*</code> product paths and city paths including <code>/*/bangalore/</code> . Grants named AI crawlers full access, but our crawler is not one of them.	DISALLOWED	Partnership or manual

Furlenco	21 disallow rules covering product category collections — sofa, bedroom, dining, storage and workspace listing paths.	DISALLOWED	Partnership or manual
Payrentz	Disallows /productlist/* , /product-detail/* , /category/* , /page/* and /tag/* — effectively the entire catalogue.	DISALLOWED	Partnership or manual
Rentzy, Cozi Rental, Furnish Rent, Adams, Fabrento	Not yet verified. Bengaluru serviceability and catalogue depth also unconfirmed.	UNVERIFIED	Gate before any adapter work

CONSEQUENCE FOR PHASE 1

Three of the five proposed MVP providers are crawlable today; two are not. Building the MVP as specified would either produce two empty columns or require ignoring the providers' own crawl directives. Recommended: ship on RentoMojo, Rentickle and Guarented, open partnership conversations with Cityfurnish and Furlenco in parallel, and give both a manual adapter so their columns are populated and honestly labelled from day one.

A comparison site with three real providers is credible. One with five columns, two of them fabricated or unlawfully obtained, is not.

Extraction route per provider

Each provider’s Bengaluru pages were fetched on 25 Aug 2026 to establish whether prices exist in the server response or are painted by JavaScript. This decides the adapter’s cost, not just its shape.

PROVIDER	DISCOVERY	PRICES IN SERVER HTML	ADAPTER	E
RentoMojo	Per-city sitemap at /sitemap/bangalore ; product paths under /bangalore/furniture/rent-...	YES names and monthly prices present in the response	HTTP fetch + HTML parse	L
Guarented	sitemap.xml carries clean per-city product URLs	NO client-rendered	Playwright	M
Rentickle	City directories only	NO city context	Playwright	L

REMARKS	Only directories only, /catalog , /_next and query-string URLs disallowed	NO City context resolved in JS — paths render as /undefined/... without it	Playwright, narrow permitted path set	
Cityfurnish, Furlenco, Payrentz	—	—	Manual sheet; partnership pursued in parallel	C h c

INFRASTRUCTURE CONSEQUENCE — REVERSED

On a paid always-on server, two Playwright adapters would have meant sizing a container for browser memory and paying for it around the clock. In a GitHub Actions runner that cost is zero: the runner is a full Ubuntu box with 4 vCPU and enough memory, it exists for eight minutes twice a day, and standard runner minutes are free within quota. The expensive option became the free one.

What Playwright still costs is *fragility*, not money. A browser adapter has more ways to break silently than an HTML one, which is why FR-6.4 treats a zero-product result as a failure rather than an empty catalogue.

Compliance gate — every provider, before an adapter is written

- Read the provider's terms of use; record the date and the relevant clauses in the provider record.
- Fetch and evaluate robots.txt for the exact paths the adapter would read.
- Prefer an API or feed; send the partnership request before writing a scraper.
- Identify ourselves in the User-Agent with a contact URL. No spoofing a browser or a named AI crawler.
- Rate-limit conservatively; a 12-hour TTL means a handful of requests per provider per day, not a crawl.
- **Never** bypass CAPTCHA, authentication, paywalls, anti-bot measures or any access control. A provider that blocks us is a provider we stop reading.
- Store only what comparison needs; attribute the source and link users to the original listing.

- Legal sign-off recorded per provider before that adapter ships. This document is not legal advice, and the crawl posture above is one input to that review, not a substitute for it.

The adapter interface

```
public interface RentalProvider {
    Provider getProvider();

    // What this adapter can actually do, so the engine can degrade honestly
    ProviderCapabilities getCapabilities();           // hasApi, supportsDeposit,
                                                    // supportsTenurePricing, isManual

    List<RentalProduct> fetchProducts(String city, RentalCategory category);

    ProviderRefreshResult refresh(String city);      // never throws to the caller;
                                                    // failures are returned as data
}
```

FR-6.1	Adapters are isolated: separate thread pool, per-provider timeout and circuit breaker. A hung Playwright session cannot stall the fleet.
FR-6.2	refresh() returns a result object carrying counts, warnings and errors. Exceptions are caught at the adapter boundary and converted; the orchestrator never sees a raw provider exception.
FR-6.3	Adapters emit raw provider payloads to the normalizer. They do not write to the price store directly and know nothing of the comparison engine.
FR-6.4	An adapter that returns zero products where it previously returned many is treated as a failure, not as "provider has no stock". Sudden coverage collapse is the primary signal that a page structure changed.
FR-6.5	Swapping a scraper for an official API must be a change to one class and its config, with no change to normalization, pricing or comparison code.

Matching products comparable

Without normalization there is no product, only a list of links. Providers name the same object four different ways:

"Single Door Fridge" "Single Door Refrigerator"
"190L Refrigerator" "Samsung 192L Fridge"
↓
REFRIGERATOR_SINGLE_DOOR · capacity_band: 150-200L

Canonical products carry structured attributes — category, size, storage, material, brand, model, capacity — because comparison happens on attributes, not on strings. A queen bed with storage and a queen bed without are different rows.

Pipeline

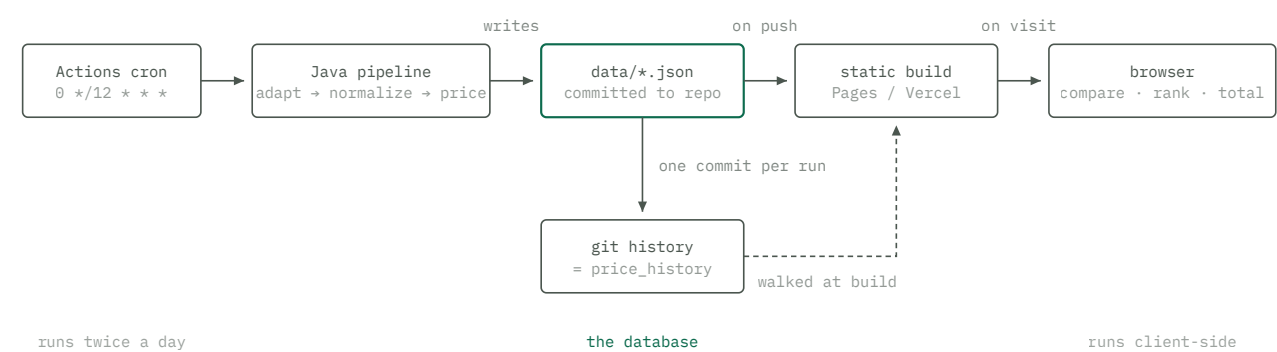
1. **Extract** — adapter returns the provider's raw fields verbatim, including the original name and URL.
2. **Parse** — deterministic rules pull capacity, size, door count, load type and brand out of the name and spec block.
3. **Match** — a rules-plus-fuzzy matcher proposes a canonical product. High-confidence matches auto-link.
4. **Review** — anything below the confidence threshold lands in an admin review queue. Unmatched items are never guessed into a comparison row.
5. **Persist** — the accepted mapping is stored in `provider_products` so it is applied automatically on every subsequent refresh.

ESTIMATE HONESTLY

Normalization, not scraping, is where this project will spend its unplanned weeks. Budget for a review queue and a human in it from day one. Auto-matching at 100% is not a realistic goal; auto-matching at 85% with a fast review UI is.

A pipeline, not a server

The modular monolith survives — same modules, same boundaries, same ArchUnit rules. What changes is its entry point: instead of `SpringApplication.run` serving HTTP forever, it is a `CommandLineRunner` that executes once, writes files, and exits. Spring Boot is still doing the work; it just stops waiting for requests.



Nothing in this picture runs continuously, and nothing in it has a bill. The two components that would have cost money — a server and a database — are replaced by a cron trigger and the repository itself.

Where each module ends up

MODULE	FATE	RUNS
provider/ , scraper/	Unchanged	In the Actions runner, twice a day
product/ (normalizer)	Unchanged	Same run
pricing/	Unchanged, and now more important — it fully resolves every cost so the client never has to	Same run
refresh/	Mostly deleted. TTL → cron, lock → concurrency , queue → the run itself	—
comparison/	Splits: cost resolution stays in Java, basket selection moves to TypeScript	Browser
admin/	Becomes a generated status page and a pull-request workflow (§19)	Build time
common/	Unchanged	—

THE ONE RULE THAT MAKES THE SPLIT SAFE

Java resolves **every** cost figure ahead of time — monthly, deposit, fees and estimated total for each of the six tenures, per provider-product. The browser then only ever does two operations: pick the minimum, and add. No business logic is duplicated across languages, because the client has none. If the browser is ever seen doing arithmetic more complicated than `min` and `sum`, a rule has leaked out of `pricing/` and needs to go back.

```
rental-comparator/
├─ .github/workflows/
│   ├── scrape.yml           cron + concurrency group + commit step
│   └─ keepalive.yml         defeats the 60-day workflow disable (§02, C-2)
├─ pipeline/                 Java 21 · Spring Boot · CommandLineRunner
│   ├── provider/ scraper/ product/ pricing/ common/
│   └─ src/test/             golden-file fixtures per adapter
├─ data/                     the database
│   ├── prices.json          resolved costs, all tenures
│   ├── catalogue.json       canonical products + attributes
│   ├── mappings.json        provider listing → canonical product
│   ├── manual/*.yaml        hand-maintained providers
│   └─ runs.json             per-provider outcome of the last run
└─ web/                      Next.js static export → Pages
```

NFR-A1	pricing/ must not import from provider/ or scraper/ . ArchUnit test in CI, as before.
NFR-A2	pricing/ stays a pure function — no I/O, no clock, no randomness — which is what lets its whole output be serialized to a file.
NFR-A3	The pipeline is a plain CLI with no dependency on GitHub. <code>./run.sh</code> on any machine produces the same <code>data/</code> output, which is the escape hatch if the free tier ever changes.
NFR-A4	The web build must fail if <code>data/</code> is malformed or a record is missing provenance. A broken build is a visible failure; a build that silently ships bad data is not.

The repository is the database

PostgreSQL is gone. The tables it would have held become JSON files that the pipeline writes and the site reads — and because they are versioned, three problems solve themselves for free.

Price history

`git log -p data/prices.json` is the full time series, timestamped and attributable, from the first run onward. No table, no migration, no retention policy.

Auditability

Every price change is a diff with an author and a commit message. “When did this jump ₹100?” is a `git blame`.

Rollback

A bad scrape that mangles the catalogue is one `git revert` away from being off the site.

FILE	WRITTEN BY	SHAPE
data/catalogue.json	Normalizer	Canonical products: id, category, brand, model, size, attributes
data/mappings.json	Normalizer + human review	provider listing → canonical product, with name, URL and match confidence. Reviewed changes arrive by pull request.
data/prices.json	Pricing module	One record per (listing, tenure): monthly, deposit, delivery, installation, other, discount, resolved total, availability, <code>scrapedAt</code> , <code>providerUrl</code>
data/manual/*.yaml	A human, by hand	Same fields, for the three providers we may not crawl. <code>scrapedAt</code> is the commit date, so a neglected sheet goes stale visibly.
data/runs.json	Every run	Per provider: started, finished, status, products found, error, coverage vs. previous run
data/pending-matches.json	Normalizer	Listings below the confidence threshold, awaiting review. Never reaches the site.

Constraints, now enforced by the build

- A price record missing providerUrl , provider or scrapedAt fails the build. Attribution was a NOT NULL ; it is now a schema validation step.
- Money stays integer paise. No floats in the pipeline or in the JSON.
- Costs are pre-resolved for all six tenures at write time, so the client never computes a total.
- Files are written sorted and pretty-printed with a stable key order — otherwise every run produces a meaningless whole-file diff and the git-as-history trick stops being readable.
- A run that would delete more than a set share of an existing provider’s products fails instead of committing. Catastrophic diffs are the signature of a broken adapter, and the point of using git is that this is checkable before it ships.

WHERE THIS DESIGN RUNS OUT

This works because the dataset is small — a few hundred products, a few hundred kilobytes. It stops working somewhere around tens of megabytes, or when two writers need to touch the data at once, or when the site needs to query rather than load. None of those are on this project’s path. If one ever is, the migration is a real database and an import script, and the pipeline above does not change.

18 – CONTRACT

Data contract, not an API

There are no endpoints, because there is no server. The site loads static JSON and does the rest in the browser. That inverts one thing worth being explicit about: the whole price dataset is public and downloadable by anyone who opens devtools. For this project that is acceptable — arguably a feature — but it must be a decision rather than a discovery.

ROUTE	SERVES

/data/catalogue.json	Canonical products and attributes, loaded once
/data/prices.json	Resolved costs for every listing and tenure
/data/runs.json	Pipeline health, powering the freshness banner and status page
/data/history/{productId}.json	Price points for one product, generated at build from git history
/bangalore/{category}-on-rent	Pre-rendered category page (§22)
/status	Generated pipeline status page (§19)

Comparison, filtering, sorting and basket selection all run client-side over the loaded data. At a few hundred products this is instantaneous and needs no index — the p95 latency requirement from the original design is satisfied by there being no network call to make.

FR-8.1	Split <code>prices.json</code> per category if it passes roughly 300 KB uncompressed, so a visitor comparing fridges does not download the sofas.
FR-8.2	Data files are content-hashed at build so a new deploy invalidates the cache; a visitor must never see yesterday’s prices with today’s freshness label.
FR-8.3	The query state stays in the URL, so a comparison remains shareable with no server to store it.

19 — OPERATIONS

Admin without an admin panel

An admin dashboard with login, roles and mutation endpoints was the other thing the server was carrying. Both of its jobs have cheaper homes, and neither needs authentication because neither writes anything a stranger could reach.

Monitoring → a generated status page

`/status` is rendered at build time from `runs.json`. It is public, read-only, and has nothing to protect.

PROVIDER	LAST SUCCESS	STATUS	PRODUCTS	DETAIL
RentoMojo	2h ago	OK	182	—
Guarented	2h ago	DEGRADED	93	Coverage down 38% vs. previous run
Rentickle	26h ago	FAILING	—	Selector returned no nodes — last 2 runs
Cityfurnish	5d ago	STALE	41	Manual sheet not updated

Layout illustration. Figures are not real.

Review queue → a pull request

When the normalizer cannot confidently match a listing, the run opens a pull request adding the proposed mapping to `mappings.json`. Reviewing means reading a diff and clicking merge; the next run picks it up. Manual price entry is the same motion — edit a YAML file, commit.

This is a better review tool than the dashboard would have been: it has history, comments, blame and revert built in, it works from a phone, and it costs nothing.

FR-7.1	Every run writes per-provider outcome, product count and coverage delta against the previous run.
FR-7.2	A failed run fails the workflow, so GitHub sends its standard failure email. That is the entire alerting system.
FR-7.3	Alert conditions beyond hard failure: coverage drop past threshold, a provider stale past 24h, or a price change outside a plausibility bound — a ₹599 bed at ₹59 is a parse bug, not a sale.
FR-7.4	Unmatched listings open a pull request; they never reach the site unreviewed.
FR-7.5	Disabling a provider is a flag in config plus a commit — its column disappears on the next build, with no deploy pipeline to wait for.
FR-7.6	The status page is linked from the site footer. If the pipeline has been dead for a week, a visitor can find that out

For a week, a visitor can find that out.

20 — NON-FUNCTIONAL

Non-functional requirements

AREA	REQUIREMENT
Latency	Comparison is instant — a static file load and an in-memory <code>min</code> . The original 500 ms p95 target is met by there being no request to make.
Frontend	Interactive within 2.5 s on a mid-range Android over 4G. Data payload under 300 KB uncompressed per category.
Partial failure	One provider failing degrades one column with an explicit label. A failed adapter never overwrites good data (FR-5.4).
Pipeline runtime	A full run finishes inside 15 minutes, comfortably under the Actions job timeout and inside the monthly minute budget.
Politeness	A handful of requests per provider per run, twice a day. Exponential backoff on 429/5xx; the run fails loudly rather than retrying into a block.
Accessibility	WCAG AA. The comparison matrix uses real <code><table></code> semantics with scoped headers.
Mobile	Card layout below 900px. No horizontal page scroll at 360px.
Observability	Run logs live in the Actions run history — free, searchable, retained. <code>runs.json</code> carries the structured summary.
Testing	Golden-file tests per adapter against captured HTML fixtures, so a markup change fails CI rather than the site. Property tests on the pricing function.
Portability	The pipeline runs identically on a laptop. No GitHub-specific code outside the workflow YAML.

21 — SECURITY

Security, mostly by subtraction

Most of the original security section protected things that no longer exist. There is no server to attack, no admin login to brute-force, no database to inject into, and no user data to leak — the site collects nothing and stores nothing. What is left is short and real.

- **Untrusted strings.** Product names come from third-party websites. They are the one genuine injection vector left: escape on render, and never pass scraped HTML through `dangerouslySetInnerHTML` .
- **Schema validation at build.** Malformed scraped data should fail the build, not reach the page. This doubles as the integrity check in §17.
- **Repository secrets.** No provider credentials in the pipeline at all — we read only public pages. If that ever changes, credentials go in Actions secrets, never in `data/` or the committed config.
- **Private repository.** Keeping the repo private hides the adapter internals and the scraped dataset. A public repo makes them a portfolio piece and gives unlimited Actions minutes; both are defensible, and the choice should be made deliberately rather than by default.
- **Workflow permissions.** The scrape workflow needs `contents: write` and nothing more. Pin third-party actions to a commit SHA rather than a tag.
- **Outbound links** carry `rel="noopener noreferrer"` .

The threat that actually applies to this project is not an attacker — it is publishing a wrong number with confidence. §13 and §17 are the security-relevant sections.

22 — GROWTH

SEO, and the moment money changes things

SEO

A statically generated site is the best possible shape for this. Each page below is pre-rendered at build from the same data the comparison uses, with real prices and a visible last-checked timestamp.

/bangalore/furniture-on-rent

/bangalore/sofa-on-rent

/bangalore/bed-on-rent	/bangalore/rentomojo-vs-guarented
/bangalore/mattress-on-rent	/bangalore/2bhk-furniture-rental
/bangalore/refrigerator-on-rent	/bangalore/bachelor-room-setup
/bangalore/washing-machine-on-rent	/bangalore/cheap-furniture-rental

No page is generated without data behind it. A thin page with an empty table is worse than no page. Since the site rebuilds twice a day, every page is as fresh as the data — which is a genuine advantage over the providers’ own static landing pages.

Revenue, and why it is out of scope

The moment an affiliate link appears, three things change at once, and it is worth seeing them together before deciding it is worth it:

WHAT CHANGES	CONSEQUENCE
Vercel Hobby no longer applies	Their terms restrict Hobby to non-commercial personal use. Either \$20/mo for Pro, or host on Cloudflare Pages, which has no equivalent clause. Building on Cloudflare Pages from day one keeps this door open for free.
The crawl posture changes character	Reading a provider’s public pages for a hobby project and doing it to earn commission are not the same conversation with that provider. §14’s compliance gate would need revisiting per provider.
Neutrality needs defending	Sponsored status must still carry zero weight in any ranking or “cheapest” calculation, sponsored results must be labelled, and commercial relationships disclosed. Easy to write, harder to hold once one provider pays and another does not.

DECISION

No revenue in any phase. The project stays non-commercial, which keeps every free tier valid, keeps the provider conversation simple, and removes the only real threat to the integrity rule. The one design choice made now to preserve optionality is hosting on Cloudflare Pages rather than Vercel.

The goal is the whole system built properly, not a demo — so the phases keep the full architecture and stage it by provider count and feature depth rather than by cutting corners.

Phase 0 — The skeleton that proves the loop

One provider, one category, end to end. This exists to prove the pipeline shape before any adapter effort is sunk into it.

AC-0.1	Actions cron runs the Java CLI, scrapes RentoMojo refrigerators, writes data/prices.json , commits, and the site rebuilds — unattended, twice in a row.
AC-0.2	The site shows those prices with a live-computed age.
AC-0.3	Killing the adapter mid-run leaves the previous committed data intact (FR-5.4).
AC-0.4	Total spend: ₹0. Actions minutes used are recorded as a baseline.

Phase 1 — MVP

Providers: RentoMojo (HTML), Guarented and Rentickle (Playwright); Cityfurnish, Furlenco and Payrentz as manual YAML sheets, labelled as such.
Categories: bed, mattress, refrigerator, washing machine.

AC-1.1	Four items and a 12-month tenure produce a populated matrix with at least 2 providers priced per row.
AC-1.2	Cheapest cross-provider basket and cheapest single-provider basket both shown, with the delta.
AC-1.3	Every price shows monthly, estimated total for the tenure, deposit separately, and a checked-at timestamp.
AC-1.4	Every price links to the provider’s own listing.
AC-1.5	All cost figures are resolved in Java; the browser only picks minima and sums. Verified by inspection of the client bundle.
AC-1.6	/status is live and linked from the footer.

AC-1.7	A manual audit of 20 sampled prices against provider sites matches at ≥ 97%.
AC-1.8	Compliance record complete per shipped provider: terms reviewed, robots checked, route recorded, dated.
AC-1.9	Keepalive workflow in place and verified against the 60-day disable.

Phase 2 — Depth

All 15 categories · filters and sorting · price history from git · Build my setup · pincode serviceability · SEO pages · the pull-request review queue.

AC-2.1	Price history is generated by walking git history. It renders only where real collected points exist — no backfill, no interpolation. An empty chart reading “collecting since <date>” is the correct output.
AC-2.2	“Lowest in the last 90 days” appears only once 90 days of runs exist.
AC-2.3	An unmatched listing opens a pull request and stays off the site until merged.
AC-2.4	Data files stay under the payload budget, or are split per category.

Phase 3 — If it is still fun

Natural-language search · multi-city · price alerts.

AC-3.1	Natural-language search converts free text to a structured query and hands it to the existing engine. The model never computes or estimates a price.
AC-3.2	Price alerts have no free serverless home worth the complexity. If built, they are a second Actions workflow that diffs the last two commits of <code>prices.json</code> and sends an email — not a subscription system.
AC-3.3	Adding a second city requires config and serviceability data only, no code change.

Risks

RISK	IMPACT	MITIGATION
The server silently stops. GitHub	High	The classic failure of an unattended hobby